Convolutional Neural Networks

Why Not Just Use a Regular (Dense) Neural Network for Images?

A plain fully-connected (dense) network treats every pixel as an independent feature:

- A small 224×224 RGB image = `224 × 224 × 3 = 150,528` input values.
- A first hidden layer with just 1,000 neurons would need `150,528 × 1,000 = 150 million` weights — for ONE layer.
- Worse: a dense layer has **no concept of spatial structure**. If you shift an object 5 pixels to the right, every single weight sees completely different inputs, so the network has to re-learn the same pattern separately for every possible position.

CNNs fix both problems with two core ideas:

Problem	Dense Network	Convolution Layer
Parameter count	Explodes with image size	Small, fixed-size filters reused everywhere (parameter sharing)
Spatial awareness	None — flattens image, loses position info	Preserves 2D structure, filters slide across the image
Translation invariance	Must relearn pattern at every position	A filter that detects an edge detects it anywhere in the image

Convolution Layers

A **convolution** slides a small learnable grid of numbers (the **filter** or **kernel**) across the image, computing a dot product at every position. Each filter learns to detect one specific pattern — an edge, a corner, a texture, a color blob.

```
Input (5x5)          Filter (3x3)         Output (3x3) "Feature Map"
1 1 1 0 0             1 0 1
0 1 1 1 0     *       0 1 0      =       [result of sliding
0 0 1 1 1             1 0 1                dot products]
0 0 1 1 0
0 1 1 0 0
```

At each position, the filter overlaps a patch of the image, multiplies element-wise, and sums the result into ONE output pixel. Then the filter slides over (by a **stride**) and repeats.

Key terms:

Term	Meaning
Kernel/Filter	Small weight matrix (e.g., 3×3, 5×5) that slides over the image
Stride	How many pixels the filter moves each step. Stride 1 = dense overlap, Stride 2 = skips every other position (shrinks output faster)
Padding	Adding zeros around the image border so the filter can also center on edge pixels. `"valid"` = no padding (output shrinks). `"same"` = pad so output size = input size
Feature Map / Activation Map	The output of applying one filter across the whole image
Channels	Depth of the input (3 for RGB) or output (= number of filters used)
Receptive Field	The region of the original input that influences one output value — grows with each conv layer stacked

Output size formula:

```
output_size = floor((input_size - kernel_size + 2*padding) / stride) + 1
```

Pooling Layers

Pooling **downsamples** feature maps — shrinking their spatial size while keeping the most important information. It has no learnable weights; it's a fixed summarization operation.

Type	Operation	Effect
Max Pooling	Takes the MAX value in each window	Keeps the strongest activation (most common choice)
Average Pooling	Takes the AVERAGE value in each window	Smoother, less common in hidden layers
Global Average Pooling	Averages the ENTIRE feature map into 1 number	Used right before the final classification layer in modern CNNs

Why pool at all? 1. **Reduces computation** — fewer pixels for later layers to process. 2. **Reduces overfitting** — fewer parameters downstream. 3. **Adds translation invariance** — small shifts in the input barely change the pooled output, since we only keep the max/average of a region. 4. **Grows the receptive field** faster, so deeper layers "see" a bigger chunk of the original image.

```
Input (4x4)              Max Pool (2x2, stride 2)       Output (2x2)
1  3  2  4                 max(1,3,5,6)=6   max(2,4,1,0)=4
5  6  1  0          →                                  →   6  4
2  1  0  3                 max(2,1,8,9)=9   max(0,3,4,7)=7      9  7
8  9  4  7
```

Feature Extraction

A CNN is really a **feature extractor** followed by a **classifier**

Convolution and pooling layer acts as feature extractor and finally extracted features are feed into Fully Connected layer for classification:

The key insight: features become more abstract with depth.

Layer Depth	What filters typically learn to detect
Early layers (close to input)	Edges, colors, simple textures, gradients
Middle layers	Combinations of edges → corners, curves, simple shapes, textures
Deep layers	Object parts (eyes, wheels, windows), high-level patterns
Final layers	Whole-object concepts (this looks like “cat”, “car”, “dog”)

Image Classification Basics

The end-to-end image classification pipeline:

```
1. INPUT IMAGE        → raw pixels (H x W x 3)
2. PREPROCESSING      → resize, normalize pixel values, data augmentation
3. FEATURE EXTRACTION → Conv + ReLU + Pool blocks (stacked)
4. FLATTEN/POOL       → turn 2D feature maps into a 1D feature vector
5. CLASSIFIER HEAD    → Dense layer(s) → raw scores (logits)
6. SOFTMAX            → convert logits into class probabilities
7. PREDICTION         → argmax(probabilities) = predicted class
```

Standard preprocessing steps (always do these):

Step	Why
Resize to fixed size (e.g., 224×224)	CNNs with dense layers need a fixed input size
Normalize pixel values (0–255 → 0–1, or standardize with mean/std)	Keeps gradients stable, speeds up training
Data augmentation (flips, crops, rotations, color jitter)	Artificially expands training data, reduces overfitting
Train/Val/Test split	Measures true generalization, catches overfitting

Evaluation metrics:

Metric	Formula	When to use
Accuracy	correct / total	Balanced classes
Precision	TP / (TP + FP)	Minimize false positives (e.g., spam detection)
Recall	TP / (TP + FN)	Minimize false negatives (e.g., cancer detection)
F1-score	2·(P·R)/(P+R)	Balance of precision & recall, imbalanced classes
Confusion Matrix	grid of predicted vs actual	See exactly which classes get confused